

Automated Research Pipeline Integration

This document outlines the steps to integrate a fully automated research pipeline into our existing backtesting repository. It describes the new modules to add, the changes to existing files, and the overall architecture needed to ingest natural language hypotheses and research documents, convert them into structured representations, generate and evaluate trading models, and deploy selected strategies into production.

1. Pipeline Overview

The pipeline has four main stages:

1. **Hypothesis ingestion and parsing** – Accept a natural language trading thesis and convert it into structured trading rules, feature definitions, and parameter search spaces using a fine-tuned language model.
2. **Document ingestion and graph/embedding creation** – Accept research documents (PDF, Word, or web links), extract text, summarise for human review, and convert to machine-readable formats: embeddings and a financial knowledge graph based on a formal ontology.
3. **Model generation, backtesting and evaluation** – Generate candidate strategies, run them through the existing backtester under realistic costs and slippage, perform robustness checks, and select models that meet risk and performance criteria.
4. **Deployment and monitoring** – Package approved models for live trading, deploy them with a staging phase, and monitor performance to trigger re-training when necessary.

2. New Modules to Add

Create a new package in the repository, `pipeline`, with the following modules:

hypothesis_parser.py – Wraps the fine-tuned LLM used to convert natural language hypotheses into structured strategy definitions. Provide a function `parse_hypothesis(thesis: str) -> ParsedHypothesis` that returns a data structure containing the instrument universe, entry/exit rules, indicators, feature list, and parameter ranges.

document_ingestion.py – Handles ingestion of PDF, Word, or web URLs. Use libraries such as `pdfplumber` for PDFs, `python-docx` for Word documents, and `requests`/`BeautifulSoup` for web pages. Provide functions:

- `extract_text(source: Union[str, Path]) -> str`
- `summarise_text(text: str) -> str` using the LLM for human-readable summary.
- `embed_text(text: str) -> np.ndarray` using sentence transformers or an embedding model.
- `build_knowledge_graph(text: str) -> nx.DiGraph` that populates a financial knowledge graph based on an ontology (e.g., FIBO). Use rule-based or model-based extraction to identify entities (companies, instruments, events) and relations (e.g., supplier relationships, regulatory obligations).

knowledge_graph.py – Defines the ontology schema and provides functions to create, query, and update the knowledge graph. Represent the graph as a NetworkX `DiGraph` or use a graph database if available. Provide helper functions to extract exposures and relationships for use in feature engineering.

****`model\_generation.py`**** – Implements the autoresearch loop. Given a ``ParsedHypothesis``, produce candidate strategies by mutating parameters, adding/removing features, and varying rule logic. Provide functions to iterate through these variants.

****`evaluation.py`**** – Wraps the existing backtester to run each candidate strategy. It should:

- Perform out of sample evaluation, cross validation, walk forward analysis, and correlation tests.
- Aggregate metrics such as Sharpe ratio, maximum drawdown, turnover, and correlation to other strategies.
- Provide gating functions to determine whether a candidate passes all thresholds.

****`deployment.py`**** – Contains helper functions to package the selected strategy into a deployable component. Provide a function ``deploy_model(model_config: dict)`` that writes configuration files, registers the model in a registry, and optionally triggers paper trading.

3. Modifications to Existing Files

****Backtester integration**** – Refactor your existing backtester module to expose a programmatic API, for example ``run_backtest(strategy_config: dict) -> BacktestResult``. The new ``evaluation.py`` will import and call this function.

****`requirements.txt`**** – Add dependencies for text extraction and knowledge graph processing (e.g., ``pdfplumber``, ``python-docx``, ``beautifulsoup4``, ``networkx``, ``sentence-transformers``).

****`README.md`**** – Update documentation to describe the new pipeline, how to ingest hypotheses and documents, and how to run the pipeline via a CLI or script.

****CLI or script**** – Add a new script (e.g., ``scripts/run_pipeline.py``) that ties together all stages. The script should:

1. Read a hypothesis from input or a file.
2. Read optional research documents and ingest them via ``document_ingestion``.
3. Call ``hypothesis_parser.parse_hypothesis`` to get a structured representation.
4. Generate candidate models via ``model_generation`` and evaluate them via ``evaluation``.
5. Deploy the best candidate via ``deployment``.
6. Output a report summarizing the candidates tested and selected strategy.

4. Integration of the Ontology Layer

To incorporate the ontology layer, define classes in ``knowledge_graph.py`` that map FIBO concepts (financial instruments, parties, contracts, events) to graph nodes. When ``document_ingestion.build_knowledge_graph`` runs, it should create edges based on relationships identified in the document. Use the graph to derive features, such as:

Exposure of a company to a supplier in a risky region.

Sector or industry classification links.

Regulatory obligations attached to instruments or entities.

Expose functions like ``get_exposures(entity_id: str)`` and ``get_related_events(entity_id: str, window: timedelta)`` to query the graph.

5. Example Pipeline Usage

```
from pipeline.hypothesis_parser import parse_hypothesis from
pipeline.document_ingestion import extract_text, summarise_text, embed_text,
build_knowledge_graph from pipeline.model_generation import
generate_candidates from pipeline.evaluation import evaluate_model from
pipeline.deployment import deploy_model # ingest and parse a hypothesis
hypothesis = "Buy tech stocks when their 20day moving average crosses above
the 50day moving average after a positive earnings surprise" parsed =
parse_hypothesis(hypothesis) # ingest research documents text =
extract_text("research.pdf") summary = summarise_text(text) embedding =
embed_text(text) kg = build_knowledge_graph(text) # integrate graph
information into features (not shown) # generate candidate models for
candidate in generate_candidates(parsed): result = evaluate_model(candidate)
if result.passes_all_gates(): deploy_model(candidate) break
```

6. Deliverables

This document is provided as a PDF (see attached) for your convenience. Ensure that your development branch includes:

- The new `pipeline` package with modules described above.

- Integration into the existing backtester via `evaluation.py`.

- Updated `requirements.txt` and `README.md`.

- A CLI script (`scripts/run\_pipeline.py`) demonstrating end-to-end execution.